

AI ASSISTED LAB ASSIGNMENT-6.1

NAME: D.Rishwanth

BATCH:14

ROLL_NO:2503A52395

Task-1: (AI-Based Code Completion for Loops)

Prompt:

“Generate Python code to print all even numbers between 1 and N
Using a loop.”

Input:

#Generate Python code to print all even numbers between 1 and N using a loop.

```
N = int(input("Enter a number: "))
print("Even numbers between 1 and", N, "are:")
for i in range(1, N + 1):
    if i % 2 == 0:
        print(i)
```

output:

```
PS C:\Users\rishwanth\.vscode> & "C:\Users\rishwanth\.vscode\Python314\python.exe" "c:/Users/rishwanth/.vscode/python.py/lab.py"
Enter a number: 10
Even numbers between 1 and 10 are:
2
4
6
8
10
PS C:\Users\rishwanth\.vscode> 
```

Task-2: (AI-Based Code Completion for Loop withConditionals)

Prompt:

“Generate Python code to count how many numbers in a list are even

and odd.”

Input:

```
def count_even_odd(numbers):
```

```
    even_count = 0
```

```
    odd_count = 0
```

```
    for num in numbers:
```

```
        if num % 2 == 0:
```

```
            even_count += 1
```

```
        else:
```

```
odd_count += 1
```

```
return even_count, odd_count
```

```
# Example usage
```

```
numbers_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
even_count, odd_count = count_even_odd(numbers_list)
```

```
print(f"Even numbers count: {even_count}")
```

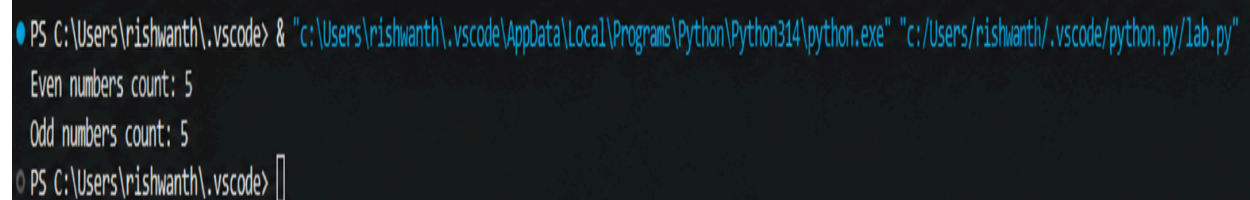
```
print(f"Odd numbers count: {odd_count}")
```

output:

Task-3: (AI-Based Code Completion for Class Attributes Validation)

Prompt:

“Generate a Python class User t

A terminal window with a dark background. The first line shows a command: "PS C:\Users\rishwanth\.vscode> & "c:\Users\rishwanth\.vscode\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/rishwanth/.vscode/python.py/lab.py"". The second line shows the output: "Even numbers count: 5". The third line shows the output: "Odd numbers count: 5". The fourth line shows the prompt: "PS C:\Users\rishwanth\.vscode>".

```
PS C:\Users\rishwanth\.vscode> & "c:\Users\rishwanth\.vscode\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/rishwanth/.vscode/python.py/lab.py"  
Even numbers count: 5  
Odd numbers count: 5  
PS C:\Users\rishwanth\.vscode> 
```

hat validates age and email using conditional statements.”

Input:

```
class User:

    def __init__(self, name, age, email):

        self.name = name

        self.age = age

        self.email = email


    def validate_age(self):

        if self.age < 0:

            return "Age cannot be negative."

        elif self.age < 18:

            return "User is a minor."

        else:

            return "User is an adult."


    def validate_email(self):

        if "@" not in self.email or "." not in self.email:

            return "Invalid email format."

        else:

            return "Email is valid."


# Example usage:

user1 = User("Alice", 25, "alice@example.com")

print(user1.validate_age()) # Output: User is an adult.

print(user1.validate_email()) # Output: Email is valid.
```

Output:

```
PS C:\Users\rishwanth> & "C:\Users\rishwanth\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/rishwanth/.vscode/python.py/lab.py"
User is an adult.
Email is valid.
PS C:\Users\rishwanth>
```

Task-4: (AI-Based Code Completion for Classes)

Prompt:

“Generate a Python class Student with attributes (name, roll number, marks) and methods to calculate total and average marks.”

Input:

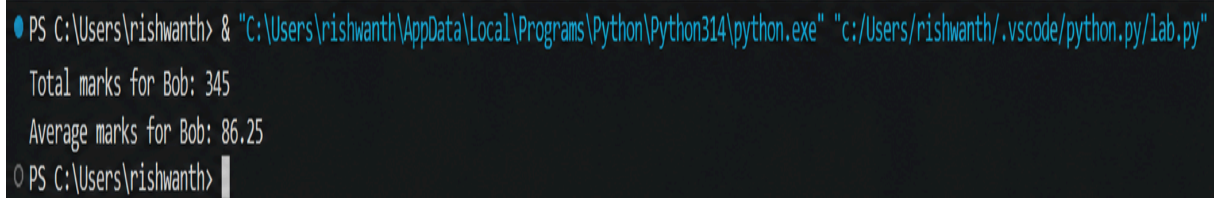
```
class Student:
    def __init__(self, name, roll_number, marks):
        self.name = name
        self.roll_number = roll_number
        self.marks = marks

    def calculate_total(self):
        return sum(self.marks)
```

```
def calculate_average(self):
    total = self.calculate_total()
    return total / len(self.marks) if self.marks else 0

# Example usage:
student1 = Student("Bob", 101, [85, 90, 78, 92])
print(f"Total marks for {student1.name}: {student1.calculate_total()}")
print(f"Average marks for {student1.name}: {student1.calculate_average()}")
```

output:

A terminal window with a dark background. The first line shows a command to run a Python script: "PS C:\Users\rishwanth> & "C:\Users\rishwanth\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/rishwanth/.vscode/python.py/lab.py"". The next two lines show the output of the script: "Total marks for Bob: 345" and "Average marks for Bob: 86.25". The final line shows the prompt "PS C:\Users\rishwanth>" with a cursor.

```
PS C:\Users\rishwanth> & "C:\Users\rishwanth\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/rishwanth/.vscode/python.py/lab.py"
Total marks for Bob: 345
Average marks for Bob: 86.25
PS C:\Users\rishwanth> |
```

Task-5: (AI-Assisted Code Completion Review)

Prompt:

“Generate a Python program for a simple bank account system using class, loops, and conditional statements.”

Input:

#Generate a Python program for a simple bank account system using class, loops, and conditional statements.

```
class BankAccount:
```

```
    def __init__(self, account_number, account_holder, balance=0):
```

```
        self.account_number = account_number
```

```
        self.account_holder = account_holder
```

```
        self.balance = balance
```

```
    def deposit(self, amount):
```

```
        if amount > 0:
```

```
            self.balance += amount
```

```
            print(f"Deposited: ${amount}. New balance: ${self.balance}.")
```

```
        else:
```

```
            print("Deposit amount must be positive.")
```

```
    def withdraw(self, amount):
```

```
        if amount > 0:
```

```
            if amount <= self.balance:
```

```
                self.balance -= amount
```

```
                print(f"Withdrew: ${amount}. New balance: ${self.balance}.")
```

```
            else:
```

```
                print("Insufficient funds.")
```

```
        else:
```

```
            print("Withdrawal amount must be positive.")
```

```
    def check_balance(self):
```

```
        print(f"Account Balance: ${self.balance}.")
```

```
        #give example usage
```

```
if __name__ == "__main__":
```

```
    # Create a new bank account
```

```
    account = BankAccount("12345", "John Doe", 1000)
```

```
# Check initial balance
```

```
account.check_balance()
```

A terminal window with a dark background and light blue text. It shows the execution of a Python script. The first line is a command prompt: PS C:\Users\rishwanth> & "C:\Users\rishwanth\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/rishwanth/.vscode/python.py/lab.py". The output consists of four lines: "Account Balance: \$1000.", "Deposited: \$500. New balance: \$1500.", "Withdraw: \$200. New balance: \$1300.", and "Account Balance: \$1300.". The prompt is followed by a cursor.

```
● PS C:\Users\rishwanth> & "C:\Users\rishwanth\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/rishwanth/.vscode/python.py/lab.py"
Account Balance: $1000.
Deposited: $500. New balance: $1500.
Withdraw: $200. New balance: $1300.
Account Balance: $1300.
○ PS C:\Users\rishwanth> |
```

```
# Deposit money
```

```
account.deposit(500)
```

```
# Withdraw money
```

```
account.withdraw(200)
```

```
# Check final balance
```

```
account.check_balance()
```

Output: